

# Using Python to Solve Linear Programs

## Lecture 1 & 2

Danko Turcic

October 8, 2026

# Agenda

- Python Basics
- Google Colab Environment
- Introduction to PuLP
- The 4-Step PuLP Framework

# Why Python for Business LPs?

- **Beyond Excel:** Handles much larger datasets.
- **Scalability:** Easy to scale from 10 variables to 10,000+.
- **Automation:** Integrates with databases and APIs.
- **Industry Standard:** Most OR and data science teams use Python.

# The Environment: Google Colab

- Cloud-based Jupyter Notebook environment.
- **Zero Setup:** Runs entirely in the browser.
- Free access to computing resources.
- Easy sharing and collaboration.

# Navigating Colab

- **Code Cells:** Where you write and execute Python.
- **Text Cells:** Where you write documentation (Markdown).
- **Execution:** Press `Shift + Enter` to run a cell.
- Variables are saved in memory after a cell runs.

# Your First Code

## The `print()` function

Used to output text to the screen.

```
print("Hello, Optimization World!")
```

- Text must be enclosed in quotes.
- Useful for debugging and viewing results.

# Variables

- A variable is a named container for storing information in memory.
- No need to declare types explicitly in Python.
- Examples:
  - `capacity = 100`
  - `product_name = "Chair"`

# Data Types

- **Integers:** Whole numbers (e.g., 10, -5).
- **Floats:** Decimal numbers (e.g., 3.14, 0.5).
- **Strings:** Text (e.g., "Optimize").
- *Why precision matters:* Floats are crucial for continuous LP variables.

# Lists (Arrays)

- Grouping ordered data together.
- Created using square brackets [].

```
products = ['A', 'B', 'C']  
costs = [10.5, 8.0, 12.2]
```

# List Operations

- **Accessing Elements:** Zero-indexed!

```
first_product = products[0] # Returns 'A'
```

- **Adding Items:**

```
products.append('D')
```

# Dictionaries (Hashmaps) - Concept

- Store data in **Key-Value** pairs.
- Like a real dictionary: look up a word (key) to find the definition (value).
- Perfect for mapping sets to parameters.

# Dictionaries - Practice

- Created using curly braces {}.

```
profits = {'A': 20, 'B': 30, 'C': 40}  
  
# Accessing a value  
profit_b = profits['B'] # Returns 30
```

# Why Objects Matter in LPs

## Python Object

- Lists (`['A', 'B']`)
- Dictionaries

## Math Concept

- Sets (Indices  $i \in I$ )
- Parameters ( $c_i, a_{ij}$ )

# What is a Python Library?

- A collection of pre-written code.
- Prevents reinventing the wheel.
- You "import" libraries to use their functions.
- Examples: Pandas (data), Matplotlib (graphs).

- An open-source linear programming library.
- Allows mathematical formulation in Python.
- Interfaces with solvers (CBC, GLPK, Gurobi, CPLEX).
- Intuitive syntax for constraints and objectives.

# Installing PuLP

- PuLP is not installed by default in Colab.
- Use the package installer pip.

```
!pip install pulp
```

- The exclamation mark (!) runs terminal commands in Colab.

# Importing PuLP

- Once installed, it must be imported into the script.

```
import pulp as pl
```

- as pl creates a short alias.
- Alternatively: `from pulp import *` (not recommended).

## The 4-Step Framework

- 1 **Initialize:** Create the model object.
- 2 **Variables:** Define decision variables.
- 3 **Objective:** Set the objective function.
- 4 **Constraints:** Add equations/inequalities.

# Step 1: Initialize the Model

- Creates the core problem object.

```
# For Maximization
model = pl.LpProblem("My_Problem", pl.LpMaximize)

# For Minimization
model = pl.LpProblem("My_Problem", pl.LpMinimize)
```

## Step 2: Define Decision Variables (Simple)

- Use `model.add_variable()`.
- Specify name, bounds, and category.

```
x = model.add_variable("x", lowBound=0, cat='Continuous')  
y = model.add_variable("y", lowBound=0, cat='Integer')
```

## Step 3: Define Objective Function

- Use the += operator to add expressions to the model.

```
# Maximize 3x + 2y  
model += 3*x + 2*y, "Objective_Function"
```

## Step 4: Define Constraints

- Also use the += operator.
- Must include <=, >=, or ==.

```
# Constraints
model += 2*x + y <= 100, "Resource_1"
model += x + 2*y <= 80, "Resource_2"
```

# Solving & Status

- Call `solve()` on the model.
- Check the status (Optimal, Infeasible, Unbounded).

```
res = model.solve()

print("Status:", res.status.name)
```

## Readability Counts

"Code is read much more often than it is written." - PEP 8

- Write code for humans, not just for the computer.
- Clear logic prevents modeling errors.

# Best Practice 1: Meaningful Names

- Avoid single-letter variables in business applications.

## Bad:

```
x = model.add_variable("x")
```

## Good:

```
chairs_produced = model.add_variable("Chairs")
```

## Best Practice 2: Using Comments

- Use # to explain *why*, not *what*.

### Bad:

```
# Add constraint  
model += x <= 50
```

### Good:

```
# Maximum warehouse capacity based on Q3 audit  
model += chairs_produced <= 50
```

# Best Practice 3: Start Simple

- **Iterative Development:** Don't build the final complex model immediately.
- Hardcode simple values first to ensure the logic works.
- Gradually replace hardcoded values with lists, dictionaries, and dynamic sets.

# The AI Trap

- **Temptation:** Asking ChatGPT to write your LP from scratch.
- **Danger:** AI often invents constraints or hallucinates math logic.
- If you don't know the fundamentals, you cannot verify if the AI's output is correct.

- **Proper Usage:** Use AI *after* grasping fundamentals.
- Good for:
  - Debugging syntax errors.
  - Suggesting performance optimizations.
  - Explaining specific error codes.

# Wrap-up & Q&A (Lecture 1)

- **Summary:** Python basics, Colab, PuLP 4-step framework.
- **Next Lecture:** Applying this to real business cases and scaling up with data structures.
- Questions?

## Part 2: Automation and Scalability

Moving away from hardcoded equations to dynamic, vectorized models.

# Refresher: Arrays (Lists) and Dictionaries

- **Lists** are used to store multiple items in a single variable. Perfect for storing our products or resources.

```
products = ["x1", "x2", "x3"]
```

- **Dictionaries** store data values in key:value pairs. Perfect for mapping profits or costs to our products.

```
profits = {"x1": 3, "x2": 2, "x3": 5}  
print(profits["x2"]) # Output: 2
```

# Refresher: Loops and List Comprehensions

- We can use **Loops** to automatically go through our lists.
- A **List Comprehension** is a one-line loop that generates a new list.  
This is the secret to building automated math equations!

## Example: List Comprehension

```
costs = {"A": 10, "B": 20}
items = ["A", "B"]

# Multiply each item's cost by 2
doubled = [costs[i] * 2 for i in items]
# Result: [20, 40]
```

# The Magic of `pl.lpSum()`

- In Excel, we use SUMPRODUCT. In math, we use Sigma ( $\Sigma$ ).
- In PuLP, we use `pl.lpSum()` wrapped around a list comprehension.

## Building the Objective Function automatically

Instead of hardcoding:

```
model += 3*x1 + 2*x2 + 5*x3
```

We write:

```
model += pl.lpSum([profits[p] * x[p] for p in products])
```

# Automating Variables (PuLP 4.0 Style)

- We also need an automated way to create 100 variables without typing `add_variable` 100 times.
- We use a **Dictionary Comprehension** to map our product names directly to PuLP variables!

## Creating a Dictionary of Variables

```
# x will be a dictionary where x["x1"] is a PuLP variable
x = {p: model.add_variable(p, lowBound=0) for p in products}
```

# The Two-Product Problem: Automated

```
products = ["x1", "x2"]
profits = {"x1": 3, "x2": 2}
r1_usage = {"x1": 2, "x2": 1}
r2_usage = {"x1": 1, "x2": 1}

model = pl.LpProblem("Automated_LP", pl.LpMaximize)

# 1. Automated Variables
x = {p: model.add_variable(p, lowBound=0) for p in products}

# 2. Automated Objective Function
model += pl.lpSum([profits[p] * x[p] for p in products])

# 3. Automated Constraints (Explicit limits for simplicity)
model += pl.lpSum([r1_usage[p] * x[p]
                    for p in products]) <= 100
model += pl.lpSum([r2_usage[p] * x[p]
                    for p in products]) <= 80

res = model.solve()
```

# Printing Results (The Easy Way)

- **Debugging your Math:** To check if you built your constraints correctly, just print the model. It will show the full mathematical equations!

```
print(model)
```

- **Printing all Answers:** You don't need to write custom loops to print your solved variables. You can ask the model for a list of every variable it knows about.

## The 2-Line Loop

```
for v in model.variables():  
    print(v.name, "=", v.varValue)
```

## The 1-Line Dictionary

```
print({v.name: v.varValue for v in model.variables()})
```

# The McDonald's Diet Problem

- **Goal:** Minimize the cost of eating at McDonald's.
- **Constraint 1:** Maximum 2000 Calories.
- **Constraint 2:** Minimum 50g Protein.
- **Constraint 3:** Minimum 100% Vitamin C.
- **Constraint 4:** Minimum 100% Calcium.
- This is a classic operations research problem. Let's solve it using our new automated Python structures!

# McDonald's Diet: The Data Arrays

```
items = ["Hamburger", "Big_Mac", "Nuggets", "Salad", "Pie"]

prices = {"Hamburger": 0.69, "Big_Mac": 1.99, "Nuggets": 1.99,
          "Salad": 2.05, "Pie": 0.79}

calories = {"Hamburger": 260, "Big_Mac": 560, "Nuggets": 250,
            "Salad": 35, "Pie": 260}

protein = {"Hamburger": 13, "Big_Mac": 25, "Nuggets": 15,
           "Salad": 2, "Pie": 3}

vit_c = {"Hamburger": 2, "Big_Mac": 2, "Nuggets": 2,
         "Salad": 40, "Pie": 40}

calcium = {"Hamburger": 15, "Big_Mac": 25, "Nuggets": 2,
           "Salad": 4, "Pie": 2}
```

# McDonald's Diet: The Continuous Solution

```
model = pl.LpProblem("Diet", pl.LpMinimize)

# cat="Continuous" allows fractions (e.g. 1.45 Salads)
x = {i: model.add_variable(i, lowBound=0, cat="Continuous")
      for i in items}

# Objective
model += pl.lpSum([prices[i] * x[i]
                   for i in items])

# Constraints
model += pl.lpSum([calories[i] * x[i]
                   for i in items]) <= 2000
model += pl.lpSum([protein[i] * x[i]
                   for i in items]) >= 50
model += pl.lpSum([vit_c[i] * x[i]
                   for i in items]) >= 100
model += pl.lpSum([calcium[i] * x[i]
                   for i in items]) >= 100

res = model.solve()
```

- Because the variables are *continuous*, the optimal answer will involve

# McDonald's Diet: The Integer Solution

- In the real world, you can't buy half a Big Mac.
- We fix this by changing the variable category to "Integer".

## The Magic Keyword

```
x = {i: model.add_variable(i, lowBound=0, cat="Integer")  
     for i in items}
```

- The rest of the code is perfectly identical.
- Forcing the solver to use whole numbers usually results in a slightly higher total cost, but it provides a realistic business solution.

# The 0-1 Knapsack Problem (The Hiking Dilemma)

- Sometimes a decision isn't "how many", it's "**Yes or No**".
- **Scenario:** You are going on a survival hike.
- **Constraint:** Your backpack holds a maximum of **10 lbs**.
- **Goal:** Maximize the total "survival value" of the items you pack.

| Item         | Value | Weight (lbs) |
|--------------|-------|--------------|
| Tent         | 10    | 5            |
| Stove        | 40    | 4            |
| Water Filter | 30    | 6            |
| Sleeping Bag | 50    | 7            |

# The 0-1 Knapsack Problem: Binary Solution

```
gear = ["Tent", "Stove", "Water_Filter", "Sleeping_Bag"]
values = {"Tent": 10, "Stove": 40,
          "Water_Filter": 30, "Sleeping_Bag": 50}
weights = {"Tent": 5, "Stove": 4,
           "Water_Filter": 6, "Sleeping_Bag": 7}

model = pl.LpProblem("Knapsack", pl.LpMaximize)

# cat="Binary" forces variables to act as Yes/No switches (1 or 0)
x = {g: model.add_variable(g, cat="Binary")
      for g in gear}

model += pl.lpSum([values[g] * x[g]
                   for g in gear])
model += pl.lpSum([weights[g] * x[g]
                   for g in gear]) <= 10

res = model.solve()

for g in gear:
    if x[g].varValue == 1:
        print(f"Pack: {g}")
```

- **Continuous:** Variables can be decimals (fluid, time, money).
- **Integer:** Variables must be whole numbers (people, cars, products).
- **Binary:** Variables are 1 or 0 (Yes/No, Build/Don't Build, Pack/Leave).
- Using Python dictionaries and `lpSum()` allows you to model thousands of products and constraints with just a few lines of code.